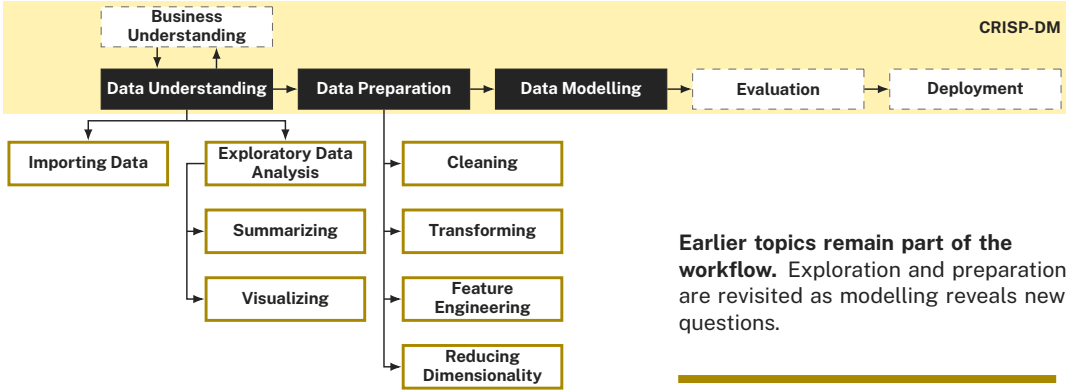
DALHOUSIE UNIVERSITY

Predictive Analytics (cont.)

Evaluation metrics, and the traps inside them

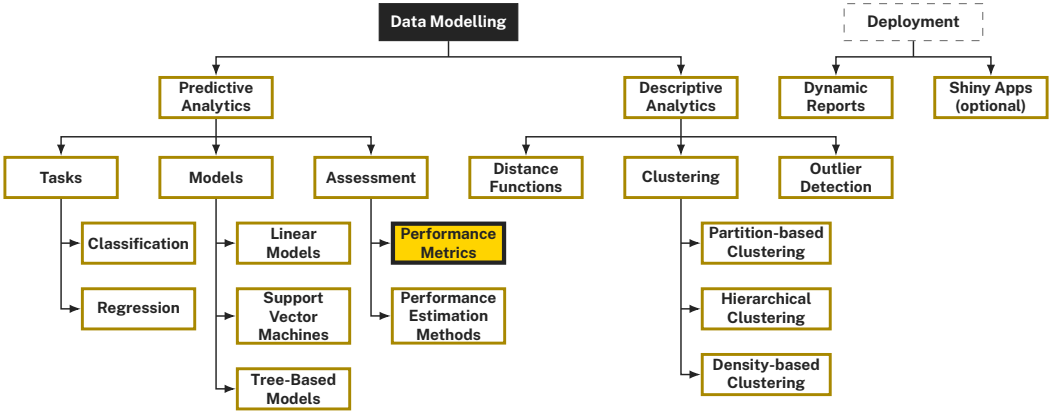
Prof. Gabriel Spadon, PhD
Faculty of Computer Science
spadon@dal.ca

The course map so far



Earlier topics remain part of the workflow. Exploration and preparation are revisited as modelling reveals new questions.

Today: performance metrics



Assessment, not modelling. Today we choose the number that judges a model, and learn what each choice quietly assumes.

Recap

One pass over the classification metrics, which make the same argument in a simpler setting.

Confusion matrix

A square $n_c \times n_c$ matrix, where n_c is the number of class values of the problem. The entries count how many times each pair (*ObservedClass*, *PredictedClass*) occurred when a classification model was tested on a set of cases.

		Pred.		
		c_1	c_2	c_3
Obs.	c_1	$n_{1,1}$	$n_{1,2}$	$n_{1,3}$
	c_2	$n_{2,1}$	$n_{2,2}$	$n_{2,3}$
	c_3	$n_{3,1}$	$n_{3,2}$	$n_{3,3}$

CLASSIFICATION METRICS

$$accuracy = \frac{n_{1,1} + n_{2,2} + n_{3,3}}{N}$$

$$error\ rate = 1 - accuracy$$

The diagonal holds the correct predictions; everything off it is an error. The error rate follows entirely from the counts in this table.

Cost-sensitive classification

Error rate assigns unit cost to every mistake and zero cost to every correct prediction. That may not be adequate for some applications.

		Pred.		
		c_1	c_2	c_3
Obs.	c_1	$n_{1,1}$	$n_{1,2}$	$n_{1,3}$
	c_2	$n_{2,1}$	$n_{2,2}$	$n_{2,3}$
	c_3	$n_{3,1}$	$n_{3,2}$	$n_{3,3}$

counts

		Pred.		
		c_1	c_2	c_3
Obs.	c_1	$B_{1,1}$	$C_{1,2}$	$C_{1,3}$
	c_2	$C_{2,1}$	$B_{2,2}$	$C_{2,3}$
	c_3	$C_{3,1}$	$C_{3,2}$	$B_{3,3}$

benefits B and costs C

Utility. Sum of the benefits minus the costs of all predictions. Models are then compared on the total balance of their predictions rather than on a count of mistakes.

Predicting a rare class

		Preds.	
		P	N
Obs.	P	TP	FN
	N	FP	TN

When one class is rare, accuracy is dominated by the majority class and stops being informative.

Precision, proportion of the signals (events) of the model that are correct.

$$Prec = \frac{TP}{TP + FP}$$

Recall, proportion of the real events that are captured by the model.

$$Rec = \frac{TP}{TP + FN}$$

Metrics for classification

DALHOUSIE UNIVERSITY

Precision and recall trade off against each other, and the F-score combines them into one number.

Precision and recall, six cases

		Preds.	
		P	N
Obs.	P	2	2
	N	1	1

Could you easily improve one at the cost of the other?

$$Precision = \frac{TP}{TP + FP} = \frac{2}{2+1} = 0.667$$

$$Recall = \frac{TP}{TP + FN} = \frac{2}{2+2} = 0.5$$

$$ErrorRate = \frac{2+1}{2+2+1+1} = 0.5$$

Predict P for every case and recall becomes 1 while precision stays at $4/6 = 2/3$. Predict P only for the single most confident case and precision can reach 1 while recall falls to 0.25.

Confusion matrix, two languages

R | caret

```r
obs  <- factor(c("P","P","P","P","N","N"))
pred <- factor(c("P","P","N","N","P","N"))

cm <- caret::confusionMatrix(
        data = pred, reference = obs,
        positive = "P")
cm$byClass[c("Precision","Recall","F1")]
# 0.6666667  0.5000000  0.5714286
```

Python | scikit-learn

```python
from sklearn.metrics import (precision_score,
    recall_score, f1_score)
obs  = ["P","P","P","P","N","N"]
pred = ["P","P","N","N","P","N"]

precision_score(obs, pred, pos_label="P")  # 0.667
recall_score(obs, pred, pos_label="P")     # 0.5
f1_score(obs, pred, pos_label="P")         # 0.571
```

Differences to know. `caret` prints *Prediction* down the rows and *Reference* across the columns, a transposed layout compared to `sklearn`, and takes the *first factor level* as positive unless you pass `positive=`. `sklearn.confusion_matrix` puts true labels on the rows in sorted label order, so `"N"` comes first and the top-left cell is TN, not TP. Both give the same three numbers once the positive class is stated. `sklearn` also returns `0.0` with a warning when nothing is predicted positive, unless you set `zero_division`.

F-score

- Precision and recall trade off.
  - Recall alone is trivial to maximize.
- Selecting a model based on two metrics is hard.
- We need a metric that captures both precision and recall.

$$F_\beta = \frac{(\beta^2 + 1) \cdot Prec \cdot Rec}{\beta^2 \cdot Prec + Rec}$$

- F_1 is the harmonic mean; F_β is a weighted harmonic mean.
- β is a user-defined value (default is 1).
- When $\beta \to 0$ the weight of *Recall* decreases.
- When $\beta \to \infty$ the weight of *Precision* decreases.

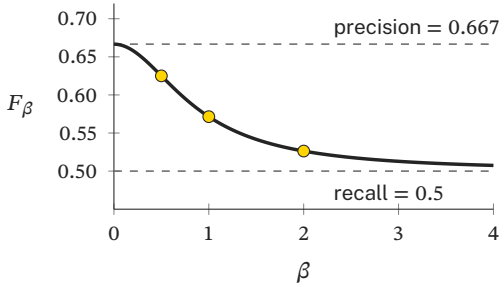
What β actually does

Same six cases, $Prec = 0.667$, $Rec = 0.5$, so $Prec \cdot Rec = 0.333$.

$$F_{0.5} = \frac{1.25(0.333)}{0.25(0.667) + 0.5} = \frac{0.417}{0.667} = 0.625$$

$$F_1 = \frac{2(0.333)}{0.667 + 0.5} = \frac{0.667}{1.167} = 0.571$$

$$F_2 = \frac{5(0.333)}{4(0.667) + 0.5} = \frac{1.667}{3.167} = 0.526$$



F_β tends to precision as $\beta \to 0$ and recall as $\beta \to \infty$. Recall has relative weight β^2 in this harmonic mean, not a fixed operational cost ratio.

Absolute error metrics

DALHOUSIE UNIVERSITY

A prediction is off by an amount. The metric decides how much that amount matters.

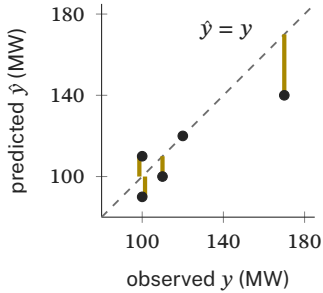
One test set, every metric

Five constructed test days. y_i is observed peak load in MW; $\hat{y}_i$ is the prediction. The regression metrics use this table; later variations are labelled.

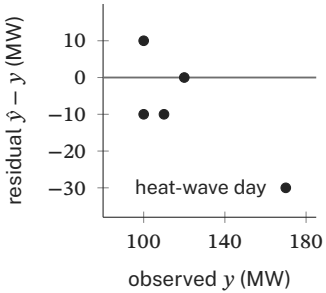
i	y_i	$\hat{y}_i$	$e_i = \hat{y}_i - y_i$	$\lvert e_i \rvert$	e_i^2	$\lvert \bar{y} - y_i \rvert$	$(\bar{y} - y_i)^2$
1	100	110	+10	10	100	20	400
2	110	100	−10	10	100	10	100
3	100	90	−10	10	100	20	400
4	120	120	0	0	0	0	0
5	170	140	−30	30	900	50	2500
Σ	600	560	−40	**60**	**1200**	**100**	**3400**

$N_{test} = 5$; $\bar{y} = 600/5 = 120$ MW. Rows 1–3 show MAPE and sMAPE calculations; row 5 is the day the model badly under-predicts.

Look before you average



Predicted against observed, with residuals in gold.



Residuals against observed; one point dominates.

Four errors of at most 10 MW and one of 30 MW. Every metric that follows is a different answer to *how much should that fifth point count?*

Mean Squared Error (MSE)

$$MSE = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} (\hat{y}_i - y_i)^2$$

where $\hat{y}_i$ is the prediction of the model under evaluation for the case i and y_i the respective true target variable value.

What are the units of MSE?
Is MSE sensitive to outliers?

ON THE RUNNING EXAMPLE

$$MSE = \frac{100 + 100 + 100 + 0 + 900}{5}$$

$$= \frac{1200}{5} = 240 \text{ MW}^2$$

The single 30 MW error contributes 900 of the 1200, **75%** of the total from **20%** of the cases. Squaring is what does that.

MSE for Fitting, RMSE for Reporting

$$MSE = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} (\hat{y}_i - y_i)^2 \qquad RMSE = \sqrt{MSE}$$

- Differentiable.
  - Makes it easier to use as the evaluation criterion to optimize a model.
- Sensitive to outliers, as it punishes larger errors more.
- Not in the same units as the Y variable.
  - Can use Root Mean Squared Error (RMSE) instead.

ON THE RUNNING EXAMPLE

$$RMSE = \sqrt{240} = 15.49 \text{ MW}$$

240 is in MW^2 and is harder to interpret operationally. RMSE is 15.49 MW, an error measure in the units of the quantity predicted.

Mean Absolute Error (MAE)

$$MAE = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} |\hat{y}_i - y_i|$$

The MAE measures the average absolute deviation between the predictions and the true values, in the same units as Y itself.

- Not differentiable at $y_i = \hat{y}_i$.
  - Harder to use as the evaluation criterion to optimize a model.
- More robust to outliers.
- Same units as variable Y.

ON THE RUNNING EXAMPLE

$$MAE = \frac{10 + 10 + 10 + 0 + 30}{5}$$

$$= \frac{60}{5} = 12 \text{ MW}$$

Is being sensitive to outliers good or bad?

What RMSE and MAE each answer

DALHOUSIE UNIVERSITY

MAE $= 12$ MW

- *On a typical day, how far off am I?*
- Every megawatt of error counts once, wherever it occurs.
- The conditional **median** minimizes MAE.

RMSE $= 15.49$ MW

- *How exposed am I to a bad day?*
- A 30 MW error costs nine times a 10 MW error.
- The conditional **mean** minimizes RMSE.

Repair the worst day only. Change $\hat{y}_5$ from 140 to 160 and leave every other row untouched:

	before	after	change
MAE	12.00 MW	8.00 MW	-33%
RMSE	15.49 MW	8.94 MW	-42%

Always $MAE \leq RMSE \leq \sqrt{N_{test}} \cdot MAE$; equality on the left only when every error is the same size. **Report both**, the gap between them *is* the outlier story.

MSE, RMSE and MAE in code

DALHOUSIE UNIVERSITY

R

```r
y <- c(100, 110, 100, 120, 170)
p <- c(110, 100,  90, 120, 140)
mean(abs(p - y))        # MAE   12
mean((p - y)^2)         # MSE   240
sqrt(mean((p - y)^2))   # RMSE  15.49193
library(Metrics)
mae(y, p); mse(y, p); rmse(y, p)   # same 3
caret::postResample(pred = p, obs = y)
#      RMSE  Rsquared       MAE
# 15.491933 0.7949126 12.000000
```

Python

```python
import numpy as np
from sklearn.metrics import (mean_absolute_error,
    mean_squared_error, root_mean_squared_error)

y = np.array([100, 110, 100, 120, 170])
p = np.array([110, 100,  90, 120, 140])
mean_absolute_error(y, p)      # 12.0
mean_squared_error(y, p)       # 240.0
root_mean_squared_error(y, p)  # 15.491933
np.sqrt(np.mean((p - y)**2))   # 15.491933
```

Absolute-Error Defaults That Bite

- `squared=False` was the old route to RMSE, deprecated in scikit-learn 1.4 and *removed* in 1.6; it now raises `TypeError`. Use `root_mean_squared_error`.
- No metric here divides by $n - 1$, but normalize by a variance and it matters. `var()` uses $n - 1$ and `np.var()` uses n, so `MSE/var(y)` is 0.282 in R and 0.353 in NumPy, and only the second is the NMSE of the next section.
- `postResample` reports `Rsquared` as the *squared correlation*, not $1 - SS_{res}/SS_{tot}$, hence 0.795 here and 0.647 from the residual definition.

Relative error metrics

DALHOUSIE UNIVERSITY

Is 12 MW a good error? The question has no answer until you say *compared with what*.

NMSE and NMAE

Divide model error by baseline error. Here the baseline uses the *test-target mean*, $\bar{y} = 120$ MW. A deployable mean baseline uses training data; compare the choices next.

$$NMSE = \frac{\sum_{i=1}^{N_{test}} (\hat{y}_i - y_i)^2}{\sum_{i=1}^{N_{test}} (\bar{y} - y_i)^2}$$

$$NMAE = \frac{\sum_{i=1}^{N_{test}} |\hat{y}_i - y_i|}{\sum_{i=1}^{N_{test}} |\bar{y} - y_i|}$$

Both are unitless. Name the baseline; a zero denominator makes either undefined.

ON THE RUNNING EXAMPLE

$$NMSE = \frac{1200}{3400} = 0.353$$

$$NMAE = \frac{60}{100} = 0.600$$

< 1 beats the baseline, $= 1$ ties it, > 1 loses to it. Here squared error is **35%** and absolute error **60%** of baseline error.

Name your baseline

Which mean? Two conventions exist for $\bar{y}$, the training-set average or the test-set average, scoring the same predictions on the same test set two different ways.

baseline $\bar{y}$	denominator	NMSE
test mean, 120	3400	0.353
training mean, 130	3900	0.308

Neither is wrong. Reporting the number without saying which one you used is.

A quantity you already know. With the test mean as the baseline,

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - NMSE = 0.647.$$

And it can go negative. A model that predicts a constant 100 MW has $SS_{res} = 5400 > 3400$, so

$$R^2 = 1 - \frac{5400}{3400} = -0.588.$$

Out of sample nothing prevents it, $R^2 < 0$ means *worse than the mean.*

R^2 is not always r^2

DALHOUSIE UNIVERSITY

RESIDUAL DEFINITION

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2} = 0.647$$

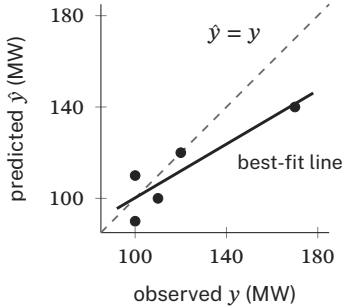
Punishes bias and mis-scaling. Used by `r2_score` in scikit-learn and by `caret::R2(form="traditional")`.

CORRELATION DEFINITION

$$r^2 = \left(\frac{\mathrm{cov}(y, \hat{y})}{s_y\, s_{\hat{y}}}\right)^2 = 0.8916^2 = 0.795$$

Blind to bias and to scale. Reported by `caret::postResample` by default.

Seeing the gap between R^2 and r^2



The points track a line of slope 0.59, not the line $\hat{y} = y$. Correlation sees only the tracking.

When do they agree? Equality is guaranteed for nonconstant ordinary least-squares fitted values *with an intercept*, scored on the training data. It need not hold on test data; here the gap is 0.15.

Mean Absolute Percentage Error

DALHOUSIE UNIVERSITY

$$MAPE = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} \frac{|\hat{y}_i - y_i|}{y_i}$$

- Undefined at a zero target; can be extremely high near zero.
- Not differentiable at $y_i = \hat{y}_i$; needs a nonsmooth optimization method.
- Here targets are positive, on a scale with a meaningful zero.
- Asymmetric, exchanging y_i and $\hat{y}_i$ yields a different MAPE.
- Unbounded for over-predictions; at most 100% for nonnegative under-predictions.

$y_i = 100,\ \hat{y}_i = 110$: MAPE $= 0.100$
$y_i = 110,\ \hat{y}_i = 100$: MAPE $= 0.091$

ON THE RUNNING EXAMPLE

$$\frac{0.100 + 0.091 + 0.100 + 0 + 0.176}{5}$$
$$= 0.0935 = 9.35\%$$

Rows 1 and 2 of the running example illustrate the key edge case.

Reading MAPE's asymmetry

THE DENOMINATOR IS THE TRUTH

With y *held fixed*, MAPE treats over- and under-prediction of the same size identically, $y = 100$ scores 0.1 for $\hat{y} = 110$ and 0.1 for $\hat{y} = 90$.

When roles are swapped, the denominator changes. The pair $\{100, 110\}$ scores $10/100 = 0.1$ one way and $10/110 = 0.09$ the other. Same error, smaller truth, bigger score.

THE REAL CONSEQUENCE

Since $\hat{y} \geq 0$, under-prediction can never exceed 100% while over-prediction is unbounded. MAPE weights absolute errors by $1/y$, so small targets count more. This can favor lower forecasts than MAE or MSE.

MAPE at a zero

DALHOUSIE UNIVERSITY

THE ZERO

Add one quiet hour with $y_6 = 0$ and $\hat{y}_6 = 5$ to the running example. The sixth term is $5/0$, so the mathematical metric is undefined; direct floating-point division returns ∞. What the software does with that row:

`Metrics::mape` (R)	`Inf`
`mean(abs((y-p)/y))`	`Inf`
`sklearn mean_absolute_percentage_error`	**3.753e15**, no error, no warning

scikit-learn replaces y_i by $\max(|y_i|, \varepsilon)$ with $\varepsilon \approx 2.2 \times 10^{-16}$. The result is finite, enormous, and easy to mistake for a real score.

Symmetric MAPE (sMAPE)

$$sMAPE = \frac{1}{N_{test}} \sum_{i=1}^{N_{test}} \frac{|\hat{y}_i - y_i|}{|\hat{y}_i| + |y_i|}$$

- Undefined if $y_i = \hat{y}_i = 0$; sensitive to small changes near zero, but bounded by 1.
- Still assumes the unit of measurement has a "meaningful zero" value.
- Fixes one type of asymmetry, exchanging y_i and $\hat{y}_i$ yields the same sMAPE.
- Asymmetric in a different way, fixing the true value and moving the same absolute deviation in different directions yields a different sMAPE.

$y_i = 100,\ \hat{y}_i = 110$: sMAPE $= 4.76\%$
$y_i = 100,\ \hat{y}_i = 90$: sMAPE $= 5.26\%$

ON THE RUNNING EXAMPLE

$$\frac{0.0476 + 0.0476 + 0.0526 + 0 + 0.0968}{5}$$

$$= 0.0489 = 4.89\%$$

Rows 1 and 2 both score $10/210 = 4.76\%$, that is the symmetry the metric buys.

Relative metrics in code

R

```r
sum((p-y)^2) / sum((mean(y)-y)^2)    # NMSE 0.3529
sum(abs(p-y)) / sum(abs(mean(y)-y))  # NMAE 0.6
library(Metrics)
mape(y, p)          # 0.09347594
smape(y, p)         # doubled convention
c(mae  = mean(abs(p-y)), mse = mean((p-y)^2),
  rmse = sqrt(mean((p-y)^2)),
  mape = mape(y, p),
  nmse = 1 - caret::R2(p, y, form = "traditional"),
  nmae = sum(abs(p-y)) / sum(abs(mean(y)-y)))

caret::R2(p, y, form = "traditional")  # 0.6470588
caret::R2(p, y)                        # 0.7949126
```

Python

```python
from sklearn.metrics import (r2_score,
    mean_absolute_percentage_error)
r2_score(y, p)                          # 0.6470588
1 - r2_score(y, p)                      # NMSE 0.3529
mean_absolute_percentage_error(y, p)  # 0.09347594

# no NMAE and no sMAPE in scikit-learn:
np.sum(np.abs(p-y)) / np.sum(np.abs(y.mean()-y))
np.mean(np.abs(p-y) / (np.abs(p) + np.abs(y)))
```

Relative-Metric Defaults That Bite

- sMAPE may divide by the sum (0–100%) or by half the sum (0–200%); `Metrics::smape` uses the doubled form.
- The explicit NMSE and NMAE use the test mean; substitute the training mean when that is the baseline you intend.
- R propagates `NA`; sklearn raises on `NaN`; NumPy returns `nan` unless you request a missing-aware mean.

Nine numbers from one test set

DALHOUSIE UNIVERSITY

Metric	Value	Units	What it answers
MAE	12.00	MW	Typical size of an error.
MSE	240.00	MW^2	Average squared error; fitting criterion.
RMSE	15.49	MW	Typical error, with large errors weighted up.
NMSE	0.353	—	Squared error as a fraction of the baseline's.
NMAE	0.600	—	Absolute error as a fraction of the baseline's.
$R^2 = 1 - SS_{res}/SS_{tot}$	0.647	—	Test-mean improvement; $= 1 - NMSE$.
r^2 (squared correlation)	0.795	—	How well predictions *track*, ignoring bias.
MAPE	9.35%	%	Error relative to each true value.
sMAPE	4.89%	%	Error relative to truth and prediction together.

The metric is part of the claim. These nine numbers summarize one test set. Choose the metric before seeing results; report its baseline and units.

What to take away

- Each metric has its own pros and cons.
- If the data set is imbalanced, accuracy is misleading.
  - You need to use precision and recall, or other metrics that do not overly focus on the majority class (sometimes called negative class).
  - Selecting a model based on two metrics at the same time may be difficult, so you can use F_1, the harmonic mean, or the weighted F_β.
- MSE is differentiable, which makes it easier to use as a model optimization criterion.
- RMSE and MAE are expressed in the same units as the target variable.
- Relative metrics ease comparison across data sets with different scales.